

PROGRAM 15

AIM

To implement a Feed Forward Neural Network (FFNN) to learn the relationship between input and output data and predict the output for a given input.

ALGORITHM

1. Start the program.
2. Prepare the input and output training data.
3. Initialize the weights and biases.
4. Pass the input values through the input layer.
5. Calculate the weighted sum and apply an activation function in the hidden layer.
6. Pass the hidden-layer output to the output layer.
7. Calculate the predicted output.
8. Calculate the error between the predicted and actual output.
9. Update the weights and biases using backpropagation.
10. Repeat the training process for the specified number of epochs.
11. Test the trained network with a new input.
12. Display the predicted output.
13. Stop the program.

PROGRAM

```
import numpy as np
X = np.array([[0,0],[0,1],[1,0],[1,1]])
Y = np.array([[0],[1],[1],[0]])
np.random.seed(1)
W1 = np.random.randn(2,4)
B1 = np.zeros((1,4))
W2 = np.random.randn(4,1)
B2 = np.zeros((1,1))
def sigmoid(x):
    return 1/(1+np.exp(-x))
def sigmoid_derivative(x):
    return x*(1-x)
```

```

for epoch in range(10000):
    H = sigmoid(np.dot(X,W1)+B1)
    O = sigmoid(np.dot(H,W2)+B2)
    error = Y-O
    dO = error*sigmoid_derivative(O)
    dH = np.dot(dO,W2.T)*sigmoid_derivative(H)
    W2 += np.dot(H.T,dO)*0.5
    B2 += np.sum(dO,axis=0,keepdims=True)*0.5
    W1 += np.dot(X.T,dH)*0.5
    B1 += np.sum(dH,axis=0,keepdims=True)*0.5
print("Predicted Output:")
print(np.round(O,2))

```

OUTPUT

```

Predicted Output:
[[0.01]
 [0.99]
 [0.98]
 [0.02]]

```

RESULT

The Feed Forward Neural Network was successfully implemented and trained using the given input data. The network learned the required input-output relationship and produced outputs close to the expected values.